

Ejercicio 12: Planificación Óptima de Viaje en Hoteles

Este ejercicio es un problema clásico de Programación Dinámica de un solo parámetro. Al igual que en problemas de justificación de textos (saltos de línea), no existe una estrategia voraz clara porque una decisión local de parar en un hotel para acercarnos a los 200 km puede penalizarnos gravemente en los días posteriores.

Debemos evaluar sistemáticamente todas las combinaciones posibles de etapas diarias mediante subproblemas solapados.

1. Qué significa "minimizar la penalización"

Iniciamos el viaje en el kilómetro 0 (que podemos considerar como el hotel 0, $a_0 = 0$) y queremos llegar al hotel n (nuestro destino a_n).

Si en un día viajamos desde el hotel j (en el kilómetro a_j) hasta el hotel i (en el kilómetro a_i), la distancia recorrida es $a_i - a_j$. La penalización de esa jornada será:

$$\text{Penalización}(j \rightarrow i) = (200 - (a_i - a_j))^2$$

Queremos encontrar una secuencia de paradas $0 = s_0 < s_1 < s_2 < \dots < s_k = n$ que minimice la suma de las penalizaciones de cada día:

$$\text{Minimizar } \sum_{d=1}^k (200 - (a_{s_d} - a_{s_{d-1}}))^2$$

2. Estado de la Programación Dinámica

Definimos el vector de estados $DP[i]$ como:

$DP[i] =$ La penalización total mínima para viajar desde el punto de partida (km 0) hasta el hotel i , parando obligatoriamente en el hotel i al final del último día.

Donde $0 \leq i \leq n$.

3. Caso Base

Para el punto de partida (kilómetro 0, antes de iniciar el viaje), la penalización acumulada es lógicamente 0:

$$DP[0] = 0$$

4. Relación de Recurrencia

Para calcular el coste mínimo para llegar al hotel i ($1 \leq i \leq n$), debemos evaluar cuál fue el último hotel j en el que paramos el día anterior. Ese hotel j puede ser cualquier parada previa desde el kilómetro 0 hasta el hotel inmediatamente anterior ($0 \leq j < i$).

El coste de venir desde j y parar en i es la penalización acumulada para llegar a j ($DP[j]$) más la penalización del viaje de ese último día de j a i :

$$DP[i] = \min_{0 \leq j < i} \left\{ DP[j] + (200 - (a_i - a_j))^2 \right\}$$

5. Pseudocódigo Modo Examen

Para dar una respuesta completa, el algoritmo no solo calcula la penalización óptima, sino que guarda un puntero `parent` para poder reconstruir hacia atrás exactamente en qué hoteles debe detenerse el profesor:

```

Funcion PlanificarViajeHoteles(a, n)
Inicio
    // 1. Añadir el origen (km 0) como el hotel 0 para unificar índices
    a_completo <- Vector de tamaño n + 1
    a_completo[0] <- 0
    Para i <- 1 Hasta n Con Paso 1 Hacer
        a_completo[i] <- a[i]
    FinPara

    // 2. Crear las estructuras DP y de reconstrucción
    DP <- Vector de tamaño n + 1
    parent <- Vector de tamaño n + 1 // Guarda el hotel previo óptimo

    // Caso Base
    DP[0] <- 0
    parent[0] <- -1

    // 3. Rellenar la tabla DP de forma iterativa
    Para i <- 1 Hasta n Con Paso 1 Hacer
        DP[i] <- Infinito

        // Probar todos los posibles hoteles previos j en los que pudimos dormir
        Para j <- 0 Hasta i - 1 Con Paso 1 Hacer
            distancia_dia <- a_completo[i] - a_completo[j]
            penalizacion_dia <- (200 - distancia_dia) * (200 - distancia_dia)
            coste_total_desde_j <- DP[j] + penalizacion_dia

            Si coste_total_desde_j < DP[i] Entonces
                DP[i] <- coste_total_desde_j
                parent[i] <- j // Registrar que es mejor venir desde j
            FinSi
        FinPara
    FinPara

    // 4. RECONSTRUCCIÓN: Recuperar la lista de paradas hacia atrás
    hoteles_parar <- Lista vacía
    nodo_actual <- n

    Mientras parent[nodo_actual] != -1 Hacer

```

```

// Añadimos el hotel (índice original 1..n) a la solución
Añadir nodo_actual al principio de hoteles_parar
nodo_actual <- parent[nodo_actual]
FinMientras

Imprimir("Penalización total mínima obtenida: ", DP[n])
Imprimir("Secuencia óptima de hoteles donde parar: ", hoteles_parar)

Retornar DP[n]
FinFuncion

```

6. Análisis de Complejidad

- **Complejidad Temporal:** $\Theta(n^2)$. El algoritmo utiliza dos bucles anidados. El bucle externo se ejecuta para cada uno de los n hoteles del mapa. Para cada hotel i , el bucle interno recorre todos los i candidatos anteriores $j \in [0, i - 1]$ calculando la penalización en tiempo constante $O(1)$. El número total de operaciones de comparación es:

$$\sum_{i=1}^n i = \frac{n(n+1)}{2} \implies \Theta(n^2)$$

La posterior fase de reconstrucción del trayecto óptimo se realiza siguiendo la cadena de padres en un tiempo lineal estrictamente acotado por $O(n)$.

- **Complejidad Espacial:** $\Theta(n)$ para almacenar los vectores unidimensionales `DP` y `parent` de tamaño $n + 1$.

7. La intuición REAL

Este ejercicio se reduce estructuralmente a encontrar el **camino de coste mínimo en un Grafo Acíclico Dirigido (DAG)**, donde:

- Los nodos son los hoteles $\{0, 1, \dots, n\}$.
- Existe un arco dirigido desde cada hotel j hacia todo hotel posterior $i > j$ cuyo peso es la penalización de viajar entre ellos: $(200 - (a_i - a_j))^2$.
- Queremos encontrar la ruta más corta desde la fuente 0 hasta el sumidero n .

Dado que el grafo está implícitamente ordenado de forma topológica de izquierda a derecha (ya que $a_j < a_i$ siempre que $j < i$), la programación dinámica lineal de coste $\Theta(n^2)$ nos garantiza encontrar el óptimo global sin necesidad de explorar combinaciones exponenciales.